

1. когда у агента есть сердцебиение - что можно с его помощью делать например
2. какие сейчас самые популярные агентские системы типа тебя (выполни поиск в интернете)

вот примеры

- openclaw
- zeroclaw
- Agent zero

- hermes agent

3. как работает память агента и как ее можно улучшить зачем она нужна какие плагины есть? какие тренды есть в мире - поищи в сети
4. куда обычно агенты сохраняют историю чатов
5. как работает векторная память и зачем это нужно нам как это использовать нам с тобой?
6. как бы мы могли записывать каждый день мои расходы доходы (книгу финансов вести) задачи ежедневный журнал с рефлексией вечером
7. какие есть варианты работы с базой данных? я вот понимаю я бы мог накидать таблицы какие то как инфоблоки в битрикс а ты бы мог очень удобно помогать мне с ними работать и это бы заменило obsidian (obsidian вообще был бы не нужен)
8. Влияет ли описание правил каких-то на агента - например запрет возможности читать
9. расскажи подробнее про это

- 🎯 Agentic workflows

- Multi-agent системы

- 🧠 Long-memory

10. а как быть с файлами если мне их тоже нужно периодически сохранять (например картинку документы)
 11. я не пойму если мы ставим какое то векторное решение в плане памяти то нам папка memory не нужна? или как это все происходит
- я не очень понимаю вообще где заканчивается структура памяти на файлах и папках
 - и где начинается векторная память
 - и можно ли обходиться только каким то одним видом памяти

12. я все больше понимаю что может это и не нужна структура папок - нужны какие-то теги структура без папок но как это может быть я не знаю

Можно ли вообще отказаться от всяких структур на компе в Obsidian и хранить как то иначе (без папок как то вообще иначе). Что бы каждый раз не ломать голову где что лежит в какой папке что бы было всех этих PARA структуризации заметок и так далее

Просто по факту атом информации (если у меня уже есть рhr значит есть) Принцип: не «где это лежит?», а «как это найти?». Система ссылок и тегов делает расположение файла неактуальным

Я не знаю как решить эту проблему. Кроме того если мы уходим от структуры папок то не понятно вот я создал папку Проккт а и мне нужно сохранить какие то связанные с этим прокутос файлы - куда это все сохранять?

13. еще раз посмотри мою идею с нодами <https://github.com/iv-litovchenko/AWN-Framework>

13.1) чем это лучше чем скиллы? есть ли вообще потребность в этом всем?

13.2) можно ли это как то развить до уровня базы данных не на файлах?

14. Что это persistent storage как настроить и для чего?

15. по итогу я понимаю что был бы такой Obsidian в интерфейсе OpenClaw мне и интерфейс Obsidian бы не нужен был (сама программа Obsidian)

16.

Я бы хотел уйти от понятия хранить файлы на компе на телефоне как это можно сделать?

17.

Смотри что мне не нравилось в tyro3 cms cms neos и других - ты создаешь запись в админке потом загружает документы фото к ней. Если запись удалить то файлы связанные с ней не удаляются это проблема но проблема даже больше в другом если я просто загружаю файлы на сервер они как бы висят в воздухе и никому не принадлежат - я делал админку на Laravel и у меня было так если я удаляю запись то удаляются и файлы вместе с ней

18.

А как вообще запускается MCP в агентах и как создать свой простой MCP

19.

Можно ли в Claude Cowork OpenClaw PoPaw и других похожих агентах подключить какой то скрипт который будет срабатывать не через ИИ

20.

Можно ли мне взять обучить кукую то простую модель ollama и обучить ее? Как происходит процесс обучения?

21.

И я не пойму что такого есть в плагинах для Obsidian что такого есть когда ты просто читаешь файлы md я тебе говорю найди что то связанное с самолета и так далее ты что с плагином Obsidian что без него через tools будешь искать точное соответствие по слову

22.

С какими базами данных агент может проще всего работать что бы вести базу данных

типа личный дневник

типа личные финансы

типа календарь

типа список задач

23.

можно ли как-то в контексте агентов использовать n8n? и чем это может быть полезным?

24.

Я не пойму если мы допустим подключили векторную память любую какую-то память - мы общались например с тобой 5 лет - обсуждали и 1000 раз меняли как вести журнал финансов к примеру

через 5 лет ты сможешь благодаря этой векторной памяти своей выдавать тот формат ведения журнала финансов который мы с тобой обсуждали 1000 раз? или нет

ведь такая память историческая != договоренности в 1 файле и сохраненная например как скил навык который мы с тобой 1000 раз поменяли как в истории гит где по итогу все равно всегда актуальная версия ведения данных

25.

Мы можем с тобой создать SQLite базу данных (например для ведения финансов) - есть какие-то навыки которые позволяют работать с этим?

могу ли я что то скачать? что бы ты тоже мог и читать и писать

Что обсуждал еще (идеи)

Самое интересное после эфира - над чем больше всего думаю сейчас

Я вот все искал какой-то подход как хранить информацию на компе - придумать какую то структуру папок еще что-то)

Не знаю сейчас еще что лучше - иметь одного универсального агента который спокойно может переключаться между любой вашей областью

- Главный (оркестратор)
- дом - агент
- работа - агент
- личная жизнь - агент
- изучение языка - агент
- канал Эда (образование) - агент

Либо же одного агента который помнит тоже что и все они и все

Мне вот чем нравится подход когда один агент одна область - например я завтра ушел с проекта - отдал этого агента и все гудбай - забыл про него

Вернулся обратно - дали доступ зашел

Очень занимает идея как именно хранить свою экосистему вообще в цифре

- есть вариант структура из папок
- есть вариант без структуры папок (это будут в основном тэги)
- есть вариант вообще делать некое подобие аналога БД

Проблема в чем - что сейчас нет универсального какого-то подхода к агентам (или я его не нашел еще) - завтра мне кто-то скинет доступ к своему агенту - у вас там так - еще кто-то у вас там так)

А был бы какой-то фреймворк - у всех бы хотя бы в плане внутренней структуры агенты бы строились на похожих принципах

Что к Васе что к Пете что к Маше зайдешь - всегда знаешь что документы лежат в этой папке - что пакеты в этой папке и так плюс минус

Хочется что бы все в плане информации было как кубики лего Вот простой пример как у меня на работе было сейчас- поступила задача- напряг мозг где создать файл- создал файл- чето в него написал (творчество человека)- закинул на яндекс диск- сделал ссылку- скинул в телеграм коллеге- она открыла телеграмм- открыла файл- прочитала

(творчество человека)- а если надо вернуться и снова все по кругуИз всего этого полезных только два шагаЭто сколько ж действий

Это намного удобнее и надёжнее, чем чистые скиллы, потому что добавляет:

- Явную иерархию доверия (схема ноды vs произвольный контент).
- Триггерный механизм — агент не сканирует всё, а реагирует на слова/события.
- Версионирование и Git — легко откатывать изменения агента.
- Гибкость: ты можешь открыть проект в Cursor/VSCode и допрограммировать сложные ноды, а редактор через чат работает только с простыми.

Как ориентироваться в 1000+ md-файлах, не умирая от контекста?

Ты прав — пихать 1000 файлов в один промпт невозможно, модель "подохнет" (токены, потеря внимания, стоимость).

Как это решается в 2026 (и как можно улучшить AWN):

- Не грузить всё. Агент сначала работает только с реестром ([AGENTS.NODES.md](#)) — это маленький индекс.
- Потом использует RAG (Retrieval-Augmented Generation): векторная база (Weaviate, Qdrant, Pinecone, или даже pgvector в Postgres) на embeddings нод и memory.
 - Чанки нод (по 300–500 токенов + overlap).
 - Hybrid search: semantic (вектора) + keyword + metadata filters (по AWN-TYPE, tags, parentNote).
 - Агент сначала делает retrieval: "найди все ноды типа Филиал с триггером 'график'".
 - Только потом загружает полные релевантные ноды + их memory в контекст.
- Можно добавить summarization: для больших memory — держать короткий summary в YAML и полный текст отдельно.
- В AWN уже есть разделение (реестр vs on_demand) — это отличная база. Можно расширить: добавить автоматическую индексацию в векторную БД при создании/обновлении ноды (скрипт на Python или через MCP-tool).

Так агент ориентируется эффективно даже на десятках тысяч файлов.

3. Хранение заметок в Obsidian без папок

Да, можно отказаться от жёсткой структуры папок. Альтернативы:

- **Теги:** помечайте заметки тегами `#тег` (например, `#php`, `#проект_X`). Поиск по тегам заменяет навигацию по папкам.
- **Ссылки:** связывайте заметки ссылками `[[Название заметки]]`. Создавайте «хаб-страницы» — центральные заметки, ссылающиеся на связанные темы.
- **Граф знаний:** используйте встроенный граф связей (Ctrl+G). Визуально видите, какие заметки связаны.
- **Поиск:** мощный поиск Obsidian (Ctrl+F, Ctrl+P) находит по содержимому, тегам, названиям.
- **Плагины:**
 - **Dataview:** запросы на языке SQL-подобного синтаксиса (например, `list from #php` покажет все заметки с тегом `#php`).
 - **Advanced URI:** автоматизируйте создание и связывание заметок.
 - **QuickAdd:** быстрый ввод и категоризация.

Принцип: не «где это лежит?», а «как это найти?». Система ссылок и тегов делает расположение файла неактуальным.

Основной принцип

Забудьте про иерархию «папка → подпапка → файл». Вместо этого:

- каждая заметка — **атом информации** (короткий фрагмент по одной теме);
- заметки связываются между собой **ссылками**;
- заметки группируются по темам с помощью **тегов**;
- поиск и навигация идут через **граф связей**, **поиск по содержимому** и **теги**.

Пошаговая схема организации

Шаг 1. Создайте единое хранилище (vault) без сложной структуры

Просто создайте хранилище в Obsidian. Не создавайте заранее папки типа «Работа», «Учёба», «Личное» и т.д. Все заметки будут лежать в одной папке (или в нескольких базовых):

- `Inbox` — для быстрых записей, черновиков, идей «на потом»;
- `Archive` — для устаревших или завершённых заметок;
- основная папка (без названия) — для всех остальных заметок.

Шаг 2. Пишите атомарные заметки

Каждая заметка должна раскрывать **одну конкретную идею или факт**. Примеры:

- хорошо: «Принцип DRY в программировании», «Рецепт панкейков», «Встреча с Петей 15.05»;
- плохо: «Мои заметки по программированию и кулинарии», «Разные мысли».

Шаг 3. Связывайте заметки ссылками

В процессе написания заметки сразу добавляйте ссылки на связанные темы с помощью синтаксиса `[[Название заметки]]`.

Пример:

В заметке «Принцип DRY» вы пишете: «Противоположность DRY — принцип WET ([WET](#))».

В заметке «Рецепт панкейков» добавляете: «Этот рецепт основан на базовом рецепте теста ([Базовое тесто для выпечки](#))».

Шаг 4. Используйте теги для категоризации

Добавляйте теги `#` в конце заметки или в начале, чтобы обозначить её тематику. Теги могут пересекаться.

Примеры тегов:

- `#программирование` , `#python` , `#рецепты` , `#выпечка` , `#встречи` , `#проект_Альфа` .

Шаг 5. Наводите порядок по мере необходимости

Не пытайтесь структурировать всё заранее. Делайте это, когда:

- в `Inbox` накопилось много черновиков — откройте их, допишите, свяжите ссылками с другими заметками, добавьте теги и перенесите в основное хранилище;
- вы заметили, что несколько заметок связаны общей темой — создайте «хаб-заметку» с названием этой темы и добавьте в неё ссылки на все связанные заметки.

Шаг 6. Пользуйтесь инструментами навигации

- **Поиск (Ctrl+P / Ctrl+F):** находите заметки по названию или содержанию.
 - **Граф знаний (Ctrl+G):** визуально видите, какие заметки связаны между собой. Ищите кластеры тем и недостающие связи.
 - **Теги:** кликните на тег `#php` в любой заметке — Obsidian покажет все заметки с этим тегом.
 - **Обратные ссылки (Backlinks):** в правой панели каждой заметки Obsidian показывает, какие другие заметки на неё ссылаются. Это помогает найти связанные идеи.
-

Пример рабочего процесса

1. Вы думаете о новом проекте. Быстро создаёте заметку в `Inbox` : «Идея: приложение для учёта книг».
2. Через неделю возвращаетесь к заметке. Дописываете детали: цели, функционал, технологии.
3. Добавляете ссылки: «Для бэкенда можно использовать [Django](#) или [Node.js](#)».
4. Добавляете теги: `#проект` , `#идеи` , `#программирование` .
5. Переносите заметку из `Inbox` в основное хранилище.
6. Создаёте новые заметки для `[[Django]]` и `[[Node.js]]` , если их ещё нет, и связываете их с заметкой о проекте.
7. Позже создаёте «хаб-заметку» `[[Мои проекты]]` и добавляете в неё ссылку на заметку «Идея: приложение для учёта книг» и другие проекты.

Принцип: не «где это лежит?», а «как это найти?». Система ссылок и тегов делает расположение файла неактуальным И как мне тогда все раскладывать какая структура ?

И какая структура может быть для заметок даже если я сделаю вывод по тегам и куда будут складываться вложенные картинки в заметки?

Структура заметок с тегами (без жёстких папок)

Базовый вариант (минимальная структура):

- `Vault/` — корневая папка хранилища;
- `Inbox/` — быстрые записи и черновики;
- `Archive/` — завершённые/устаревшие заметки;
- `Templates/` — шаблоны заметок;
- `Assets/` или `Media/` — для всех вложений (картинки, аудио, PDF и т.д.);
- основная папка (в корне `Vault/`) — для всех рабочих заметок (без дополнительной иерархии).

Как работают теги:

Теги заменяют папки. Примеры:

- `#работа` , `#личное` , `#учёба` — глобальные категории;
- `#проект_Альфа` , `#клиент_Иванов` — проекты и клиенты;
- `#php` , `#дизайн` , `#маркетинг` — темы и навыки;
- `#idea` , `#todo` , `#done` — статусы;
- `#meeting_2024_05` — временные метки.

Организация вложений (картинок и файлов)

Вариант 1. Единая папка для всех вложений

- папка: `Vault/Assets/` или `Vault/Media/` ;
- внутри можно создать подпапки по типу:
 - `Images/` ;
 - `Audio/` ;
 - `PDFs/` ;
 - `Others/` .

Плюсы: всё в одном месте, легко бэкапить.

Минусы: может стать хаотично при тысячах файлов.

Вариант 2. Вложенные папки рядом с заметками

- для заметки `Vault/Мой проект.md` создаётся папка `Vault/Мой проект_assets/` ;
- все вложения к заметке кладутся туда.

Плюсы: файлы привязаны к заметке, меньше путаницы.

Минусы: разрозненность, сложнее бэкапить всё сразу.

Вариант 3. Гибридный подход (рекомендуемый)

- основная папка: `Vault/Assets/` ;
- подпапки по **дате создания**: `Assets/2024/05/` , `Assets/2024/06/` и т.д.;
- или подпапки по **теме**: `Assets/Проекты/` , `Assets/Идеи/` , `Assets/Рецепты/` .

Это даёт баланс между порядком и гибкостью.

Практические рекомендации по организации

1. Именование файлов

Чтобы избежать дубликатов и путаницы, используйте схему именования:

- по дате: `20240515_прототип_v2.jpg` ;
- с префиксом темы: `проект_Альфа_диаграмма_архит.jpg` ;
- уникальные ID (если используете плагины типа Obsidian UID).

2. Ссылки на вложения в заметках

Синтаксис в Markdown:

- картинка: `![Описание](путь/к/файлу.jpg)` ;
- файл: `[Ссылка на PDF](путь/к/файлу.pdf)` .

Примеры:

- `![Схема базы данных](Assets/2024/05/db_schema_v1.jpg)` ;
- `[ТЗ для дизайнера](Assets/Документы/ТЗ_дизайн_v3.pdf)` .